

Tugas Praktikum Sesi 3

Pemrograman Mobile



Dosen Pengampu: Jefry Sunupurwa Asri, S.Kom, M.Kom

Disusun Oleh:

Bagas Yoas Sibagariang

(20230801254)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

TANGERANG

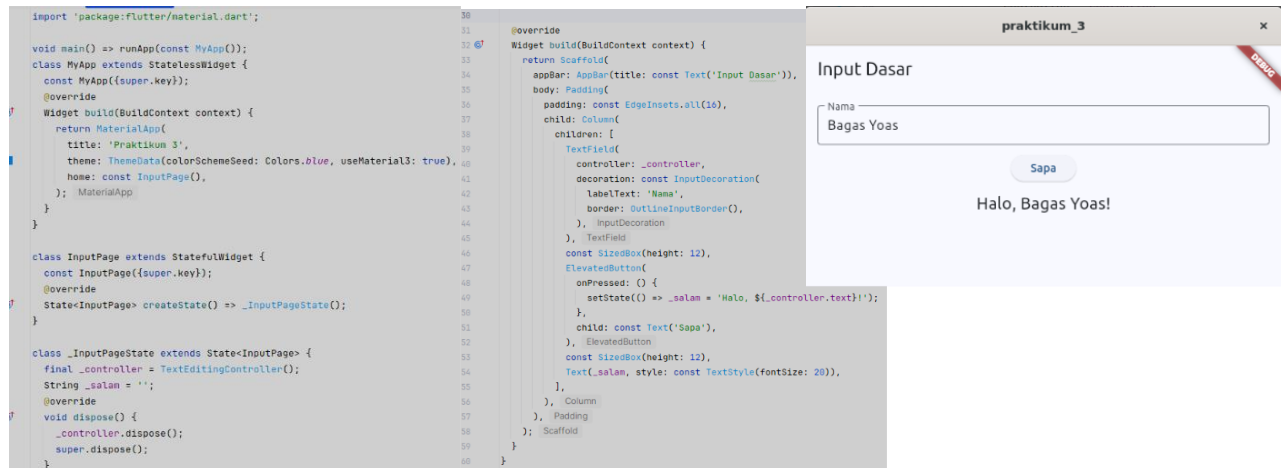
2026

1. Langkah-Langkah Praktikum

a. Input Dasar dengan TextField

Ganti isi lib/main.dart dengan kerangka berikut, lalu tambahkan halaman InputPage.

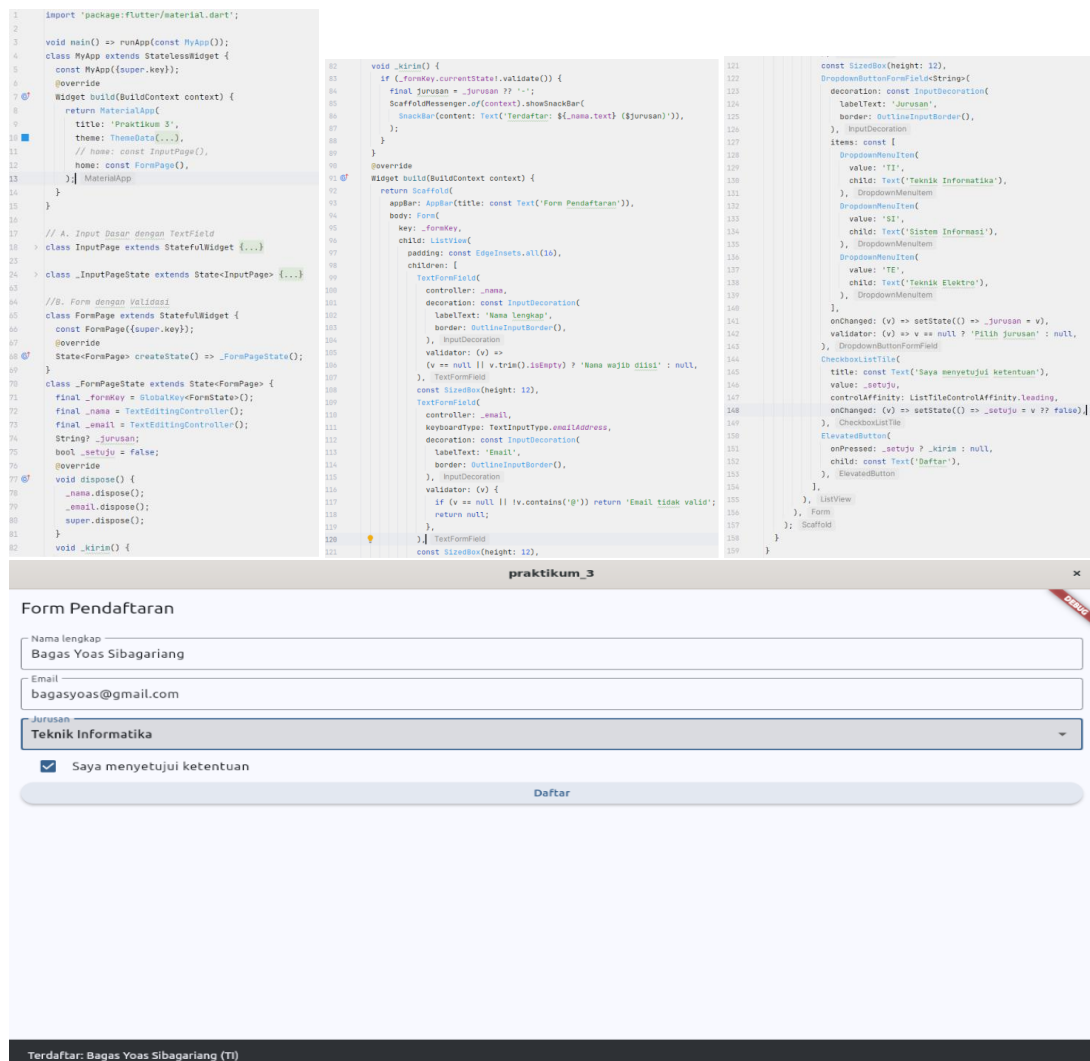
Hasil :



b. Form dengan Validasi

Buat halaman baru `FormPage` (di file yang sama), lalu ubah `home`: menjadi `const FormPage()`.

Hasil:



c. Butuh State Management.

Pada Bagian A dan B, state hidup di dalam satu halaman (setState). Bayangkan aplikasi daftar tugas: halaman daftar menampilkan tugas, sedangkan halaman tambah membuat tugas baru. Kedua halaman perlu berbagi data yang sama. Mengoper data lewat constructor dan callback dari satu halaman ke halaman lain cepat menjadi rumit. Solusinya: pindahkan data ke satu objek bersama yang dapat diakses kedua halaman. Inilah peran Provider.

d. Aplikasi Daftar Tugas dengan Provider

Langkah 1. Pasang paket pada terminal di folder proyek:

--- flutter pub add provider

```
praktikum_3 flutter pub add provider
Woah! You appear to be trying to run flutter as root.
We strongly recommend running the flutter tool without superuser privileges.
/
Resolving dependencies...
Downloading packages... (3.4s)
async 2.11.0 (2.13.1 available)
boolean_selector 2.1.1 (2.1.2 available)
characters 1.3.0 (1.4.1 available)
clock 1.1.1 (1.1.3 available)
collection 1.18.0 (1.18.1 available)
```

Langkah 2. Ganti seluruh isi lib/main.dart dengan bagian model dan entry point berikut:

```
1 import 'package:flutter/material.dart';
2 import 'package:provider/provider.dart';
3
4 // MODEL: satu tugas (Langkah 2)
5 class Tugas {
6   String judul;
7   bool selesai;
8   Tugas(this.judul, {this.selesai = false});
9 }
10
11 // STATE BERSAMA: daftar tugas + fungsi-fungsinya
12 class TugasModel extends ChangeNotifier {
13   final List<Tugas> _items = [];
14
15   List<Tugas> get items => List.unmodifiable(_items);
16   int get jumlahSelesai => _items.where((t) => t.selesai).length;
17
18   void tambah(String judul) {
19     _items.add(Tugas(judul));
20     notifyListeners(); // beri tahu widget agar tampilan diperbarui
21   }
22
23   void toggle(int index) {
24     _items[index].selesai = !_items[index].selesai;
25     notifyListeners();
26   }
27
28   void hapus(int index) {
29     _items.removeAt(index);
30     notifyListeners();
31   }
32 }
33
34 void main() {
35   runApp(
36     // Provider dipasang di ATAS MyApp supaya semua halaman bisa mengakses
37     ChangeNotifierProvider(
38       create: (_) => TugasModel(),
39       child: const MyApp(),
40     ),
41   );
42 }
43
44 class MyApp extends StatelessWidget {
45   const MyApp({super.key});
46
47   @override
48   Widget build(BuildContext context) {
49     return MaterialApp(
50       title: 'Daftar Tugas',
51       theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
52       home: const TugasPage(),
53     );
54   }
55 }
```

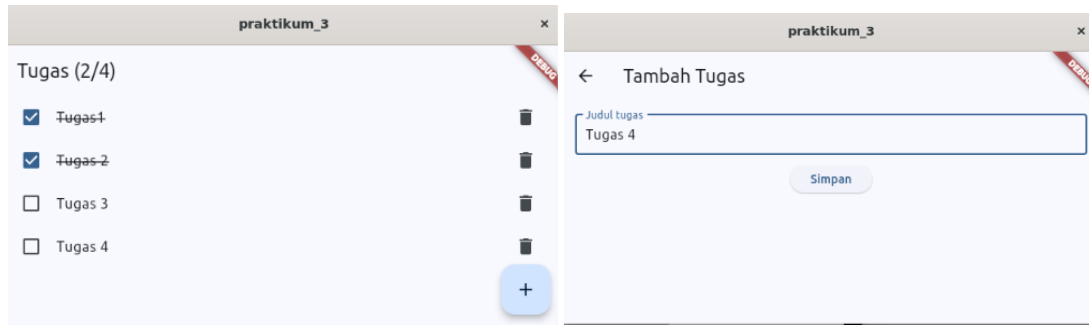
Langkah 3. Tambahkan halaman daftar. Perhatikan pemakaian context.watch di dalam build dan context.read di dalam callback.

```
57 // HALAMAN DAFTAR (Langkah 3)
58 class TugasPage extends StatelessWidget {
59   const TugasPage({super.key});
60
61   @override
62   Widget build(BuildContext context) {
63     final model = context.watch<TugasModel>(); // watch: di dalam build
64     return Scaffold(
65       appBar: AppBar(
66         title: Text('Tugas (${model.jumlahSelesai}/${model.items.length})'),
67       ),
68       body: ListView.builder(
69         itemCount: model.items.length,
70         itemBuilder: (context, i) {
71           final t = model.items[i];
72           return ListTile(
73             leading: Checkbox(
74               value: t.selesai,
75               onChanged: (i) => context.read<TugasModel>().toggle(i), // read
76             ),
77             title: Text(
78               t.judul,
79               style: TextStyle(
80                 decoration: t.selesai ? TextDecoration.lineThrough : null,
81               ),
82             ),
83             trailing: IconButton(
84               icon: const Icon(Icons.delete),
85               onPressed: () => context.read<TugasModel>().hapus(i),
86             ),
87           );
88         },
89       ),
90       floatingActionButton: FloatingActionButton(
91         onPressed: () {
92           Navigator.push(
93             context,
94             MaterialPageRoute(builder: (context) => const TambahPage()),
95           );
96         },
97         child: const Icon(Icons.add),
98       ),
99     );
100   }
101 }
```

Langkah 4. Tambahkan halaman untuk menambah tugas (halaman terpisah, tetapi memakai model yang sama):

```
103 // HALAMAN TAMBAH (Langkah 4)
104 class TambahPage extends StatefulWidget {
105   const TambahPage({super.key});
106
107   @override
108   State<TambahPage> createState() => _TambahPageState();
109 }
110
111 class _TambahPageState extends State<TambahPage> {
112   final _controller = TextEditingController();
113
114   @override
115   void dispose() {
116     _controller.dispose();
117     super.dispose();
118   }
119
120   void _simpan() {
121     final judul = _controller.text.trim();
122     if (judul.isEmpty) return;
123     context.read<TugasModel>().tambah(judul);
124     Navigator.pop(context);
125   }
126 }
127
128 @override
129 Widget build(BuildContext context) {
130   return Scaffold(
131     appBar: AppBar(title: const Text('Tambah Tugas')),
132     body: Padding(
133       padding: const EdgeInsets.all(16),
134       child: Column(
135         children: [
136           TextField(
137             controller: _controller,
138             autofocus: true,
139             decoration: const InputDecoration(
140               labelText: 'Judul tugas',
141               border: OutlineInputBorder(),
142             ),
143             onSubmitted: (text) => _simpan(),
144           ),
145           const SizedBox(height: 12),
146           ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
147         ],
148       ),
149     ),
150   );
151 }
```

Hasil :

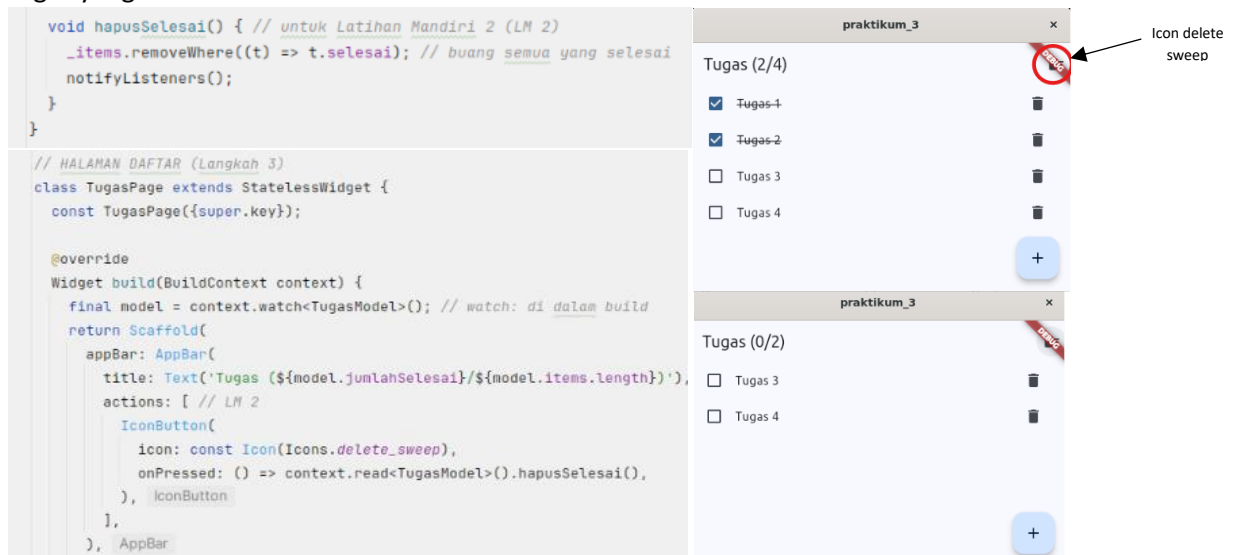


2. Latihan Mandiri

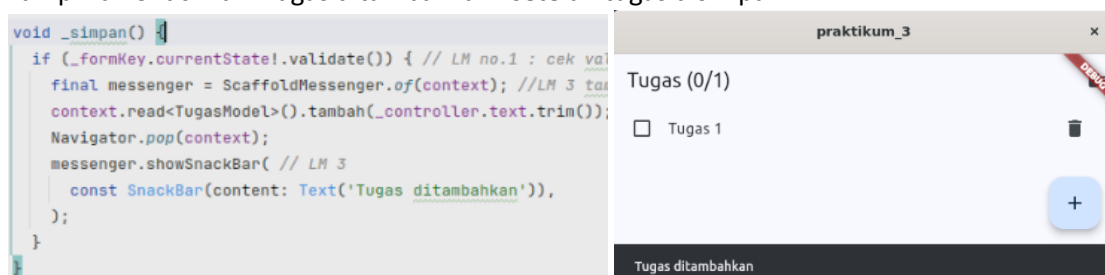
1. Tambahkan validasi pada TambahPage: judul minimal 3 karakter dan tampilkan pesan error (ubah ke TextFormField + Form).



2. Tambahkan method hapusSelesai() pada TugasModel dan tombol di AppBar untuk menghapus semua tugas yang sudah selesai.



3. Tampilkan Snackbar "Tugas ditambahkan" setelah tugas disimpan.



4. Bila daftar kosong, tampilkan teks "Belum ada tugas" di tengah layar

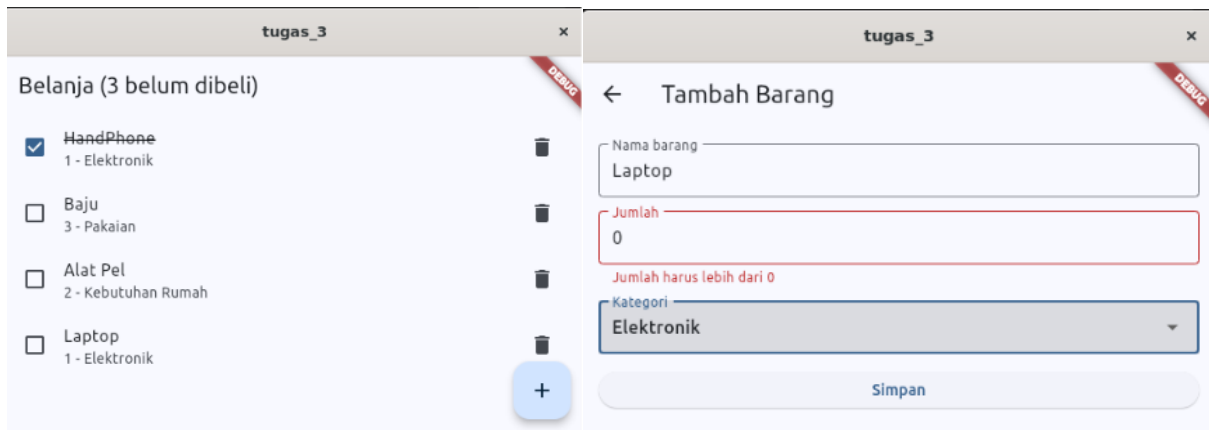


3. Tugas

Buat aplikasi Daftar Belanja dengan ketentuan:

1. Halaman Form Tambah: nama barang (wajib), jumlah (wajib, angka lebih dari 0), dan kategori (dropdown), dengan validasi pada setiap isian
2. Halaman Daftar: menampilkan semua barang, dapat dicentang sebagai "sudah dibeli" dan dihapus.
3. State disimpan pada satu ChangeNotifier dan dibagikan dengan Provider; AppBar menampilkan jumlah barang yang belum dibeli.
4. Kumpulkan: tangkapan layar kedua halaman (termasuk tampilan pesan error validasi) dan berkas main.dart (atau tautan repositori)

Hasil Tugas Sesi 3 (di folder tugas_3)



4. Jawaban Pertanyaan Refleksi

1. Mengapa TextEditingController harus di-dispose?

Controller memegang sumber daya di memori. Kalau halaman ditutup tanpa melepaskannya, objek itu tetap tersimpan (memory leak). `dispose()` melepaskannya saat widget dihancurkan.

2. Kapan cukup `setState`, dan kapan beralih ke Provider?

`setState` cukup untuk data yang hanya dipakai satu widget atau halaman, seperti checkbox atau teks sapaan. Beralih ke Provider saat data dipakai banyak halaman, seperti daftar tugas yang dibaca dan diubah di dua halaman.

3. Apa yang terjadi jika `notifyListeners()` lupa dipanggil?

Data di model berubah, tetapi tampilan tidak ikut berubah. Widget yang memakai `context.watch` hanya dibangun ulang saat model memberi sinyal lewat `notifyListeners()`.

4. Mengapa di `onPressed` memakai `context.read`, bukan `context.watch`?

`onPressed` hanya butuh memanggil fungsi sekali saat tombol ditekan, bukan memantau perubahan data. `watch` hanya valid di dalam `build()`. Memakainya di callback tidak tepat dan bisa memicu error.